

Recursividad en Python — Patrones esenciales

Mario Alejandro Bravo Ortiz

Objetivos de aprendizaje



Comprender la recursión

Entender cómo las funciones pueden llamarse a sí mismas y reconocer cuándo aplicar soluciones recursivas



Dominar tres patrones

Implementar correctamente el patrón índice, divide y vencerás, e incluir/excluir en problemas reales



Aplicar en práctica

Resolver ejercicios y reconocer qué patrón utilizar según el tipo de problema que enfrentas

Mapa de ruta del curso

1

Patrón índice

Recorrer estructuras secuenciales mediante posiciones. Ejemplos: suma de listas, máximo, invertir cadenas

2

Divide y vencerás

Partir el problema en subproblemas más pequeños. Ejemplos: búsqueda binaria, merge sort

3

Incluir/Excluir

Explorar todas las combinaciones posibles tomando decisiones binarias. Ejemplo: generar subconjuntos

4

Práctica integrada

Aplicar los conceptos con ejercicios y verificar tu comprensión con preguntas rápidas



Patrón 1: Índice

¿Qué es?

El patrón índice utiliza una posición numérica para recorrer estructuras secuenciales como listas o cadenas. En cada llamada recursiva, avanzamos o retrocedemos el índice hasta alcanzar el caso base.

Casos de uso:

- Recorrer arrays o listas
- Procesar caracteres de cadenas
- Acumular resultados secuenciales



Anatomía del patrón índice



Caso base

Verificar si el índice alcanzó el final de la estructura (o el inicio, si recorremos al revés)



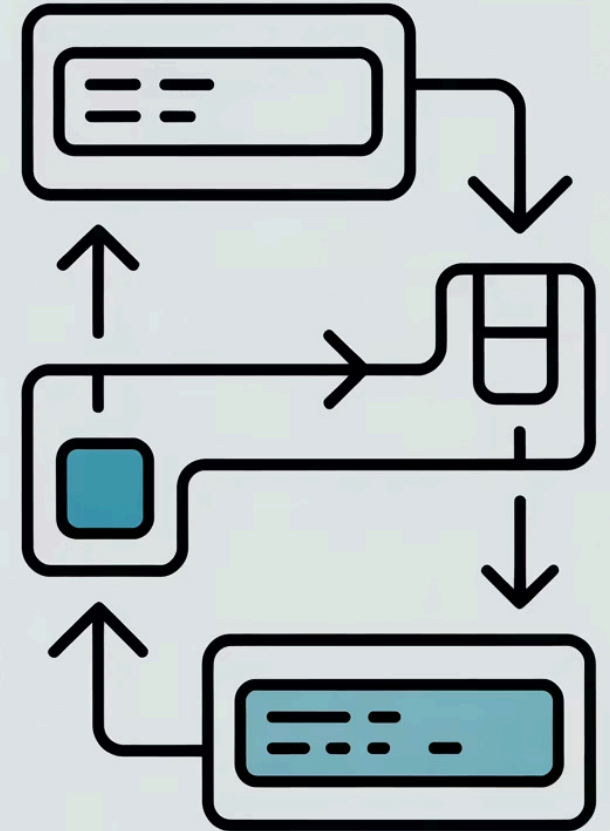
Avance recursivo

Procesar el elemento actual y llamar a la función con el siguiente índice ($i+1$ o $i-1$)



Combinar resultados

Retornar el valor procesado combinado con el resultado de la llamada recursiva



Ejemplo 1: Suma de lista

```
def suma_lista(lista, i=0):  
    """Suma todos los elementos de una lista usando índice."""  
    # Caso base: Llegamos al final  
    if i == len(lista):  
        return 0  
  
    # Paso recursivo: elemento actual + suma del resto  
    return lista[i] + suma_lista(lista, i + 1)  
  
# Uso  
numeros = [5, 3, 8, 2, 9]  
resultado = suma_lista(numeros)  
print(f"La suma es: {resultado}") # 27
```

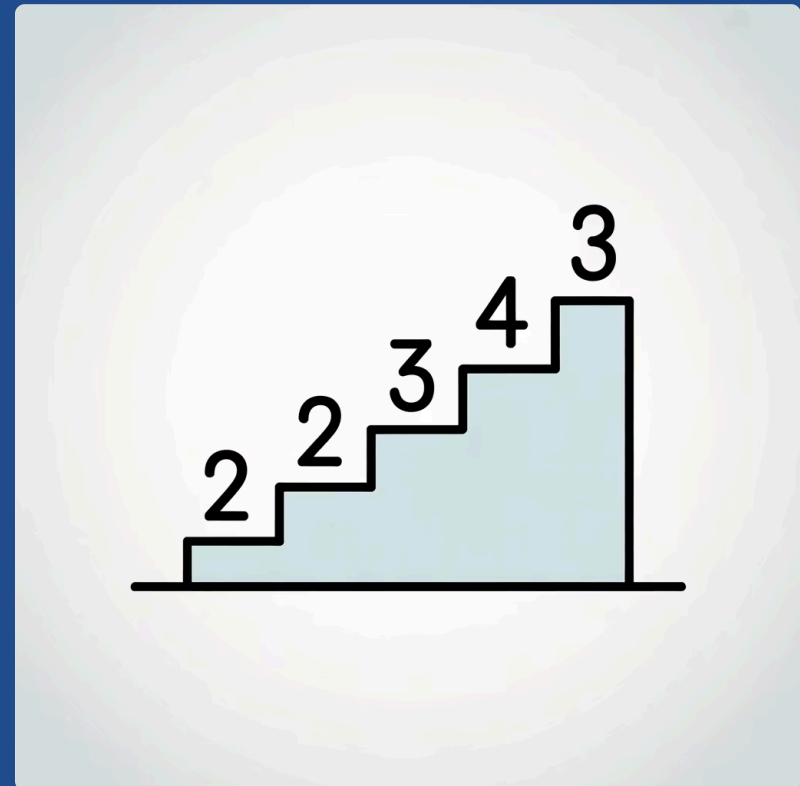
Traza de ejecución: $\text{suma_lista}([5,3,8,2,9], 0) \rightarrow 5 + \text{suma_lista}(\dots, 1) \rightarrow 5 + (3 + \text{suma_lista}(\dots, 2)) \rightarrow \dots \rightarrow 5 + 3 + 8 + 2 + 9 + 0 = 27$

Ejemplo 2: Encontrar el máximo

```
def maximo(lista, i=0):  
    """Encuentra el valor máximo."""  
    # Caso base: último elemento  
    if i == len(lista) - 1:  
        return lista[i]  
  
    # Recursión: comparar actual  
    # con máximo del resto  
    max_resto = maximo(lista, i + 1)  
  
    if lista[i] > max_resto:  
        return lista[i]  
    else:  
        return max_resto  
  
# Prueba  
valores = [12, 45, 23, 67, 34]  
print(maximo(valores)) # 67
```

Estrategia clave

En cada paso, comparamos el elemento actual con el máximo del resto de la lista. La recursión se encarga de encontrar ese máximo.



Ejemplo 3: Invertir cadena

```
def invertir(cadena, i=0):  
    """Invierte una cadena usando índice."""  
    # Caso base: índice fuera de rango  
    if i >= len(cadena):  
        return ""  
  
    # Recursión: construir desde el final  
    return invertir(cadena, i + 1) + cadena[i]  
  
# Demostración  
texto = "Python"  
resultado = invertir(texto)  
print(resultado) # "nohtyP"
```

¿**Cómo funciona?** La recursión llega hasta el final (caso base retorna ""), y al regresar, cada llamada agrega su carácter al final. Así "P" termina siendo el último carácter añadido.

Mini-ejercicio: Patrón índice

1

Contar vocales

Escribe una función recursiva `contar_vocales(cadena, i=0)` que cuente cuántas vocales (a, e, i, o, u) hay en una cadena.

2

Producto de lista

Implementa `producto_lista(lista, i=0)` que multiplique todos los elementos de una lista usando el patrón índice.

❏ **Pista:** Para contar vocales, verifica si `cadena[i].lower() in "aeiou"`. Para el producto, considera que el caso base debe retornar 1, no 0.

Patrón 2: Divide y vencerás

Este patrón divide el problema en subproblemas más pequeños e independientes, los resuelve recursivamente, y luego combina sus soluciones para obtener el resultado final.

1. Dividir

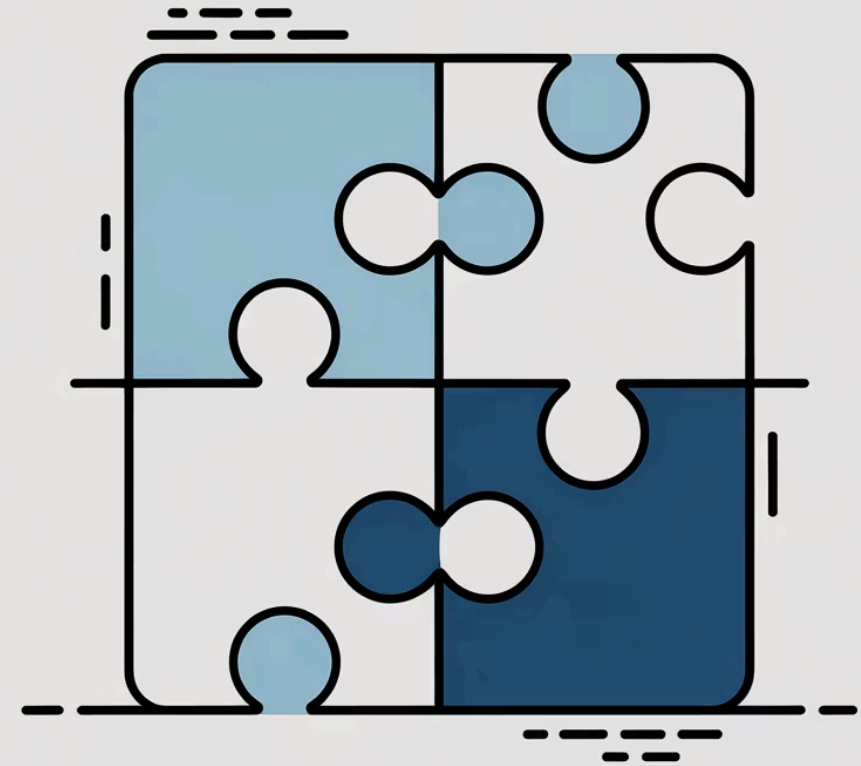
Partir el problema en dos o más partes más pequeñas

2. Conquistar

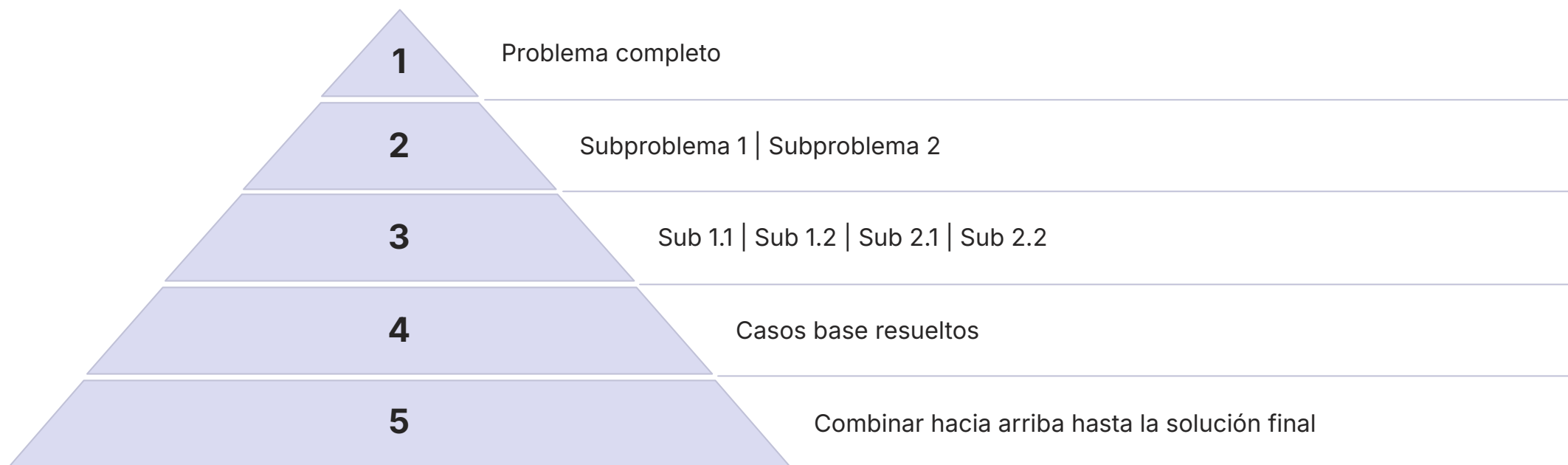
Resolver cada subproblema de forma recursiva

3. Combinar

Integrar las soluciones parciales en una solución completa



Visualización: Divide y vencerás



La clave está en que cada nivel del árbol recursivo trabaja con una porción más pequeña del problema original, hasta llegar a casos tan simples que se resuelven directamente.

Ejemplo: Búsqueda binaria

```
def busqueda_binaria(lista, objetivo, inicio=0, fin=None):
    """Busca un elemento en una lista ordenada."""
    if fin is None:
        fin = len(lista) - 1

    # Caso base: no encontrado
    if inicio > fin:
        return -1

    # Dividir: calcular punto medio
    medio = (inicio + fin) // 2

    # ¿Lo encontramos?
    if lista[medio] == objetivo:
        return medio

    # Conquistar: buscar en mitad apropiada
    if objetivo < lista[medio]:
        return busqueda_binaria(lista, objetivo, inicio, medio - 1)
    else:
        return busqueda_binaria(lista, objetivo, medio + 1, fin)

# Demostración
numeros = [2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78]
posicion = busqueda_binaria(numeros, 23)
print(f"Encontrado en índice: {posicion}") # 5
```

Ejemplo: Merge Sort

```
def merge_sort(lista):
    """Ordena una lista usando merge sort."""
    # Caso base: lista de 0 o 1 elemento
    if len(lista) <= 1:
        return lista

    # Dividir: partir por la mitad
    medio = len(lista) // 2
    izq = lista[:medio]
    der = lista[medio:]

    # Conquistar: ordenar cada mitad
    izq_ordenada = merge_sort(izq)
    der_ordenada = merge_sort(der)

    # Combinar: mezclar las mitades
    return merge(izq_ordenada, der_ordenada)

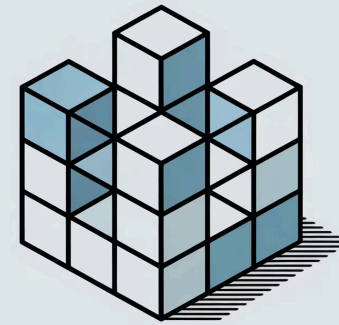
def merge(izq, der):
    """Combina dos listas ordenadas."""
    resultado = []
    i = j = 0

    while i < len(izq) and j < len(der):
        if izq[i] <= der[j]:
            resultado.append(izq[i])
            i += 1
        else:
            resultado.append(der[j])
            j += 1

    resultado.extend(izq[i:])
    resultado.extend(der[j:])
    return resultado
```

Paso a paso

1. Divide la lista por la mitad
2. Ordena recursivamente cada mitad
3. Mezcla las mitades ordenadas comparando elementos



Mini-ejercicio: Divide y vencerás

1

Potencia recursiva

Implementa `potencia(base, exponente)` usando divide y vencerás. Pista: $x^8 = (x^4)^2$, y $x^4 = (x^2)^2$.

2

Contar ocurrencias

Escribe `contar_en_rango(lista, valor, inicio, fin)` que cuente cuántas veces aparece un valor en un rango de la lista, dividiendo el rango en dos.

Patrón 3: Incluir/Excluir

Este patrón explora todas las combinaciones posibles tomando decisiones binarias: en cada paso, decidimos si incluimos o excluimos un elemento. Es especialmente útil para problemas de generación de conjuntos o combinaciones.

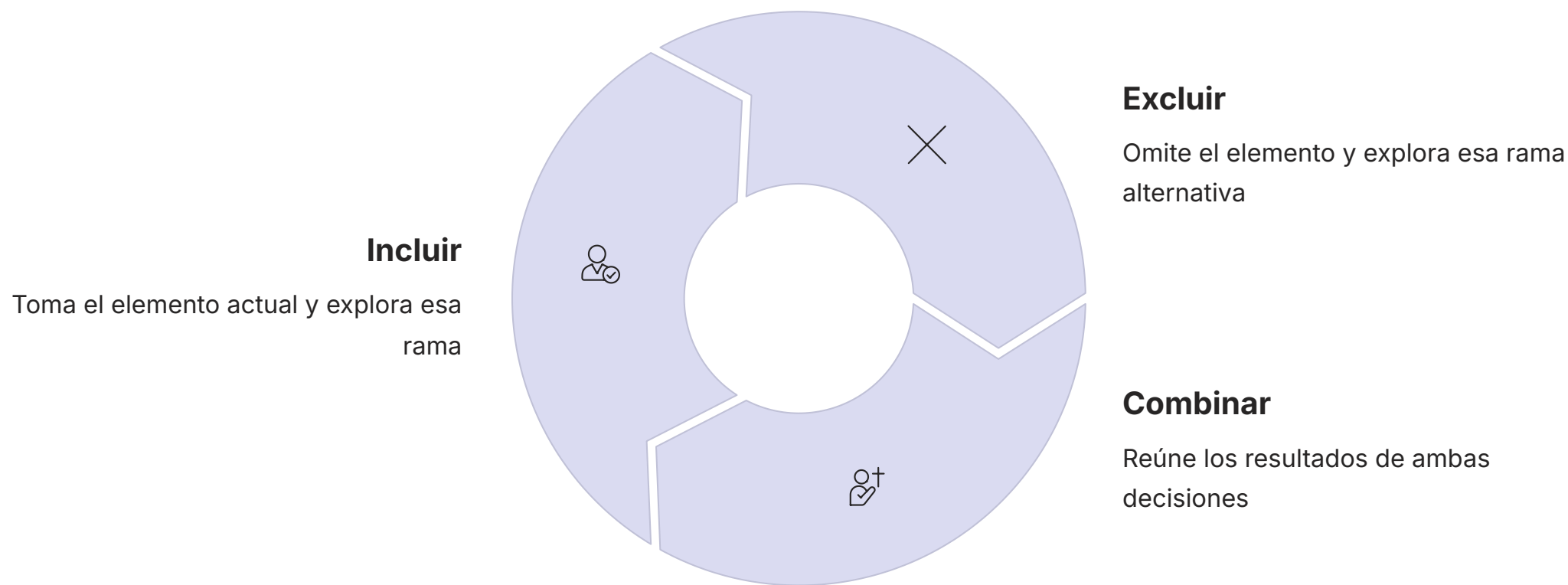


Diagrama de decisión

1

Elemento actual: X

Tenemos dos opciones en cada nivel del árbol recursivo

2

Rama 1: Incluir X

Agregamos X al conjunto actual y resolvemos para los elementos restantes

3

Rama 2: Excluir X

No agregamos X y resolvemos para los elementos restantes sin él

4

Resultado final

La unión de todas las ramas nos da todas las combinaciones posibles

Cada elemento duplica el número de posibilidades, por eso una lista de n elementos genera 2^n subconjuntos.

Ejemplo: Power Set (Subconjuntos)

```
def subconjuntos(lista, i=0):  
    """Genera todos los subconjuntos posibles de una lista."""  
    # Caso base: procesamos todos los elementos  
    if i == len(lista):  
        return [[]] # Conjunto vacío  
  
    # Recursión: obtener subconjuntos del resto  
    subs_sin_actual = subconjuntos(lista, i + 1)  
  
    # Incluir: agregar elemento actual a cada subconjunto  
    subs_con_actual = []  
    for sub in subs_sin_actual:  
        subs_con_actual.append([lista[i]] + sub)  
  
    # Combinar: unir ambas opciones (incluir y excluir)  
    return subs_sin_actual + subs_con_actual  
  
# Demostración  
conjunto = [1, 2, 3]  
resultado = subconjuntos(conjunto)  
print(f"Total de subconjuntos: {len(resultado)}")  
print(resultado)  
# [[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]
```

Mini-ejercicio: Incluir/Excluir

Suma objetivo

Escribe `puede_sumar(lista, objetivo, i=0)` que retorne True si es posible seleccionar algunos elementos de la lista para que sumen exactamente el valor objetivo.

Generar cadenas binarias

Implementa `cadenas_binarias(n)` que genere todas las cadenas de n bits (ej: para n=2, retorna ['00', '01', '10', '11']).

- ❏ **Pista para suma objetivo:** En cada paso, tienes dos opciones: incluir `lista[i]` en la suma o no incluirlo. El caso base es cuando `i == len(lista)`: verifica si objetivo es 0.

Checklist de recursión efectiva

✓ Caso base

- ¿Está claramente definido?
- ¿Se alcanza en todos los escenarios?
- ¿Retorna un valor apropiado?
- ¿Maneja casos límite (listas vacías, índices fuera de rango)?

✓ Progreso recursivo

- ¿Cada llamada se acerca al caso base?
- ¿Los parámetros cambian correctamente?
- ¿Evitamos recursión infinita?



✓ Corrección

- ¿Los resultados parciales se combinan bien?
- ¿Funciona para casos pequeños ($n=0$, $n=1$)?
- ¿Los tipos de datos son consistentes?

Verifica tu comprensión

1 ¿Qué patrón usarías para sumar los dígitos de un número?

Respuesta: Patrón índice, convirtiendo el número a cadena o usando módulo para extraer dígitos.

2 En merge sort, ¿cuál es el caso base y por qué?

Respuesta: Una lista de 0 o 1 elemento, porque ya está ordenada y no requiere más división.

3 ¿Por qué el power set de una lista de 4 elementos genera 16 subconjuntos?

Respuesta: Porque cada elemento tiene 2 opciones (incluir/excluir), entonces $2^4 = 16$.

4 ¿Qué sucede si olvidas el caso base en una función recursiva?

Respuesta: Recursión infinita que termina en un error de desbordamiento de pila (stack overflow).

5 ¿Cuándo preferirías divide y vencerás sobre el patrón índice?

Respuesta: Cuando puedes dividir el problema en partes independientes que se resuelven más eficientemente, como en búsqueda binaria u ordenamiento.

¡Felicitaciones! Has completado los patrones esenciales de recursión. Practica implementando variaciones de estos ejemplos para dominar cada patrón.